UNIVERSIDADE DE MOGI DAS CRUZES

RAFAEL SOUZA SANTANA

Pediu Patrão

Mogi das Cruzes

2026

RAFAEL SOUZA SANTANA

Pediu Patrão

Trabalho desenvolvido no curso de Bacharelado em Engenharia de Software da Universidade de Mogi das Cruzes, sob orientação do Prof. Alessandro Aparecido da Silva Horas, com o objetivo de analisar, projetar e implementar um dashboard administrativo integrado ao sistema de pedidos Pediu Patrão, incluindo o desenvolvimento da API, a integração com o banco de dados e a documentação dos requisitos, endpoints e testes realizados.

Mogi das Cruzes

2026

1. CONTEXTO E OBJETIVO

O estabelecimento precisa acompanhar as vendas, consultar a situação dos pedidos e identificar os produtos mais vendidos. O painel utiliza dados persistidos no MongoDB e disponibilizados por uma API do próprio sistema, sem manter uma base de vendas separada. O objetivo da atividade é aplicar análise e projeto de software, modelagem de dados, desenvolvimento de APIs e integração entre backend e interface. A solução usa Java 21, Spring Boot, Spring Security, Spring Data MongoDB, Thymeleaf, CoreUI e Chart.js.

A página do painel está em /dashboard e consome GET /api/dashboard/resumo. Os indicadores são atualizados ao abrir a página e quando o usuário aplica os filtros ou solicita nova atualização.

2. REQUISITOS FUNCIONAIS

Código	Requisito	Critério de aceitação
RF01	Indicadores gerais de vendas	Apresentar faturamento, total de pedidos, ticket médio, finalizados e cancelados no período.
RF02	Análise de vendas	Apresentar gráfico diário de faturamento e distribuição por situação, com dados equivalentes em texto.
RF03	Situação dos pedidos	Contar os pedidos por situação atual e considerar a compatibilidade com o estado antigo EM_PREPARO.
RF04	Produtos mais vendidos	Exibir os 10 produtos com maior quantidade vendida em pedidos finalizados elegíveis.
RF05	Filtros de consulta	Permitir datas inicial e final e situação opcional, aplicadas a todos os componentes.
RF06	Integração com a API	Carregar o painel por endpoint autenticado e autorizado, consultando o banco de dados.
RF07	Atualização e tratamento	Tratar carregamento, ausência de dados, falha de consulta, sessão expirada e dados anteriores desatualizados.

3. REQUISITOS NÃO FUNCIONAIS

Código	Requisito	Implementação
RNF01	Controle de acesso	Autorizar no filtro HTTP e nos serviços; preservar CSRF nas escritas e não retornar senhas.
RNF02	Precisão financeira	Usar BigDecimal, valores persistidos após desconto e arredondamento HALF_UP no ticket.
RNF03	Atomicidade	Gravar pedidos e clientes junto com a auditoria em transações MongoDB.
RNF04	Concorrência	Usar versão nos pedidos e bloqueio transacional dos administradores antes de remoções ou rebaixamentos.
RNF05	Desempenho	Consultar um intervalo com índice de entrada, em uma consulta por resumo, limitado a 366 dias.
RNF06	Acessibilidade	Fornecer rótulos, foco visível, mensagens anunciadas e tabelas alternativas aos gráficos.
RNF07	Responsividade	Adaptar indicadores e gráficos ao celular e permitir rolagem horizontal das tabelas.
RNF08	Manutenção	Reutilizar a arquitetura existente e concentrar as regras de negócio nos serviços.

4. PERFIS E PERMISSÕES

O enunciado atribui o painel ao responsável pelo estabelecimento, sem definir perfis técnicos específicos. A decisão de projeto foi permitir a consulta a ADMIN e GERENTE. O funcionário responsável pelo atendimento utiliza o perfil ATENDENTE.

Operação	ADMIN	GERENTE	ATENDENTE
Consultar dashboard	Sim	Sim	Não
Consultar pedidos	Sim	Sim	Sim
Registrar pedidos	Não	Sim	Sim
Alterar status e aplicar desconto	Não	Sim	Não
Cancelar pedidos	Não	Sim	Não
Consultar clientes	Não	Sim	Sim

Operação	ADMIN	GERENTE	ATENDENTE
Cadastrar alterar e excluir clientes	Não	Não	Sim
Administrar usuários e perfis	Sim	Não	Não
Consultar auditoria	Sim	Sim	Não

5. REGRAS DE NEGÓCIO

1.1. Usuários e administração

RN01. Os perfis aceitos são ADMIN, GERENTE e ATENDENTE. Senhas são armazenadas com hash e não aparecem nas respostas da API.

RN02. Um administrador não pode alterar o próprio perfil ou login nem excluir o próprio usuário. Também não pode excluir ou rebaixar o último ADMIN do banco.

RN03. Antes de atualizar ou excluir usuários, a transação bloqueia os documentos ADMIN. Isso faz operações concorrentes disputarem a mesma escrita antes de contar os administradores remanescentes.

RN04. Senha vazia na atualização mantém o hash existente. Na criação, a senha é obrigatória. O nome de usuário não pode se repetir.

1.2. Pedidos e auditoria

RN05. O pedido exige cliente existente e ao menos um item. Cada item exige produto existente, ativo e quantidade positiva. O servidor consulta o preço e registra nome, quantidade e preço unitário no pedido.

RN06. O servidor calcula o valor total e define status, data de entrada e responsável. Valores comerciais e responsáveis enviados pelo cliente não são aceitos como fonte de cálculo.

RN07. O fluxo permitido é RECEBIDO → EM_PREPARACAO → PRONTO → SAIU_PARA_ENTREGA ou RETIRADO → FINALIZADO. A saída registra horário e responsável. O serviço reconhece EM_PREPARO como estado antigo equivalente durante a transição.

RN08. Somente o GERENTE pode aplicar desconto entre 0% e 20%, antes da saída e enquanto o pedido estiver ativo. O total é recalculado sobre os preços registrados nos itens.

RN09. Somente o GERENTE pode cancelar, antes da saída, com justificativa obrigatória. O pedido cancelado mantém o histórico e não permite novas alterações.

RN10. Inclusão de pedidos, mudanças de status, descontos, cancelamentos e alterações de clientes geram auditoria persistente com usuário, perfil, data, operação, entidade e valores anteriores e novos. Não existem endpoints para alterar ou excluir a auditoria.

RN11. A operação auditada e seu evento são confirmados na mesma transação. Uma falha aborta a operação. O ambiente precisa oferecer suporte a transações MongoDB por replica set ou Atlas.

6. CONTRATO DA API DO DASHBOARD

1.3. Autenticação e acesso

Entrar em /login para obter a sessão. GET /dashboard serve a página para ADMIN e GERENTE. A API também verifica esses perfis. Sem autenticação, o Spring Security redireciona para /login. Escritas das APIs existentes exigem token CSRF; o GET de consulta não exige esse token.

1.4. Endpoint de resumo

GET /api/dashboard/resumo

Parâmetro	Tipo	Regra e padrão
inicio	Data YYYY-MM-DD	Opcional; padrão de 29 dias antes de fim.
fim	Data YYYY-MM-DD	Opcional; padrão de hoje em São Paulo.
status	Texto	Opcional; omitir para consultar todas as situações.

Valores de status aceitos: RECEBIDO, EM_PREPARACAO, PRONTO, SAIU_PARA_ENTREGA, RETIRADO, FINALIZADO e CANCELADO. Se somente

inicio for informado, fim continua sendo hoje. Se somente fim for informado, a consulta usa 30 dias encerrados nessa data.

Exemplo de requisição autenticada no console do navegador:

```
const resposta = await fetch(
  '/api/dashboard/resumo?inicio=2026-11-07&fim=2026-11-07',
  {headers: {Accept: 'application/json'}}
);
console.log(resposta.status, await resposta.json());
```

Exemplo ilustrativo de resposta 200, com dois pedidos finalizados no dia:

```
{
  "inicio": "2026-11-07", "fim": "2026-11-07",
  "status": null, "fusoHorario": "America/Sao_Paulo",
  "faturamento": 60.50, "totalPedidos": 2,
  "finalizados": 2, "cancelados": 0,
  "ticketMedio": 30.25, "semValor": 0,
  "vendas": [
    {"data": "2026-11-07", "faturamento": 60.50, "pedidos": 2}
  ],
  "situacao": {
    "RECEBIDO": 0, "EM_PREPARACAO": 0, "PRONTO": 0,
    "SAIU_PARA_ENTREGA": 0, "RETIRADO": 0,
    "FINALIZADO": 2, "CANCELADO": 0
  },
  "produtos": [
    {"produtoId": "pizza", "nome": "Pizza", "quantidade": 4}
  ]
}
```

Os campos início, fim e status representam o filtro aplicado. vendas contém uma entrada por dia, inclusive dias zerados. situação apresenta as contagens atuais. produtos contém o ranking e semValor informa a quantidade de finalizados excluídos dos cálculos por ausência de valor registrado.

1.5. Respostas e erros

HTTP	Condição	Tratamento
200	Consulta válida, inclusive vazia	Exibir o resumo ou informar ausência de pedidos.
302	Sessão ausente ou expirada	Redirecionar para login; a interface informa a expiração.
400	Data, intervalo ou situação inválidos	JSON com o campo erro; corrigir o filtro.
403	Perfil sem acesso	Negar a consulta.
503	Falha de acesso aos dados	Informar indisponibilidade e permitir nova tentativa.

Exemplos de corpos de erro:

```
{"erro":"Status inválido."}
{"erro":"Não foi possível acessar os dados. Tente novamente em instantes."}
```

Ao falhar uma atualização, a interface conserva os resultados anteriores e informa que podem estar desatualizados. A mensagem de indisponibilidade é genérica; o diagnóstico técnico deve investigar a exceção no backend.

7. TESTES E EVIDÊNCIAS

Execução registrada em 7 de novembro de 2026 com Java 21 e Maven Wrapper. Resultado da suíte automatizada: 69 testes, zero falhas, zero erros e zero testes ignorados.

Suíte	Quantidade	Verificação principal
AutorizacaoTest	31	Perfis, CSRF, formulários, criação de pedidos pelo gerente e operações restritas.
DashboardControllerTest	5	Página, autorização, filtros inválidos e erro 503 sem detalhes internos.
DashboardServiceTest	4	Cálculos, ranking, filtros, dias vazios, dados ausentes e limites de data.
UsuarioServiceTest	8	Perfis, hashes e proteção do próprio acesso e último administrador.
TransacaoAuditoriaTest	2	Rollback pelo interceptor e fronteiras transacionais dos serviços.
PedidoServiceTest	14	Cálculo no servidor, itens, status, descontos e cancelamentos.
ClienteAuditoriaTest	2	Registro das alterações de clientes.
AuditoriaMongoTest	2	Conversão BSON e inserção de eventos pelo repositório.
PedidoDashboardQueryTest	1	Limites de data combinados no mesmo campo e representação BSON correta.

8. REFERENCIAS UTILIZADAS

CoreUI para Bootstrap: https://coreui.io/bootstrap/docs/getting-started/introduction/